

uG24081616uP 8-bit Microprocessor

Engineering Specification

Rev 0.1

Authors:

Sree Ram Duvvuri

CONFIDENTIAL

Table of Contents

Revision History	6
1 Introduction	7
1.1 Features	7
1.2 Scope & Purpose	8
1.3 Abbreviations & Acronyms	8
1.4 References	9
1.5 Implementation Nomenclature	9
2 Top Level Design	10
2.1 Overview	10
2.2 Top Level Configurability	11
2.3 Top Level IO Description	11
2.4 Clock-Reset Specification	11
2.4.1 Clocks	11
2.4.1.1 Adaptive Synchronous Handshake	12
2.4.1.2 JTAG Adaptive Clocking (if configured)	13
2.4.2 Resets	13
3 Design Specification	15
3.1 Instruction Set Architecture	16
3.1.1 Instruction Set	16
3.1.2 Instruction Formats	16
3.1.2.1 12-bit Argument Instruction Format	16
3.1.2.2 10-bit Argument Instruction Format	16
3.1.2.3 0 Argument Instruction Format	16
3.1.2.4 2-fetch Instruction Format	17
3.1.2.5 2-operands Instruction Format	17
3.1.2.6 Data Transfer & Stack Instruction Format	17
3.1.2.7 Arithmetic & Logical Instruction Format	17
3.2 Instruction Pipeline Staging	17
3.2.1 Instruction Fetch (IF)	18
3.2.2 Instruction Decode (ID)	19
3.2.3 Execute (EX)	19
3.2.4 Write Back (WB)	19
3.3 Pipeline States-Actions	19
3.4 Bus Interface Unit (BIU)	20
3.5 Instruction Fetch Unit (IFU)	21
3.5.1 Instruction Buffer (IBUF)	21
3.6 Instruction Decoder (IDEC)	21
3.7 Execution Unit (EU)	21
3.7.1 Arithmetic and Logical Unit (ALU)	21
3.7.2 Control Unit (CU)	22
3.8 Write Back	22

3.9	GPRs, SFRs & CSRs.....	22
3.10	JTAG Debug.....	22
3.11	Reset Strapping.....	22
3.12	Performance Counters.....	22
4	Appendix	23
4.1	Integration Guidelines	23
4.2	Programming Guidelines.....	23
4.3	Implementation Guidelines.....	23
4.4	Memories	23
4.5	Macros (Hard IPs).....	23
4.6	Performance Analysis	23
4.7	Estimates	23
	Open Issues	24

List of Figures

Figure 2.1 uG24081616uP System Overview	10
Figure 2.2 Adaptive Synchronous Handshake	12
Figure 2.3 Adaptive Clocking scheme.....	13
Figure 3.1 uG24081616uP Microarchitecture diagram.....	15
Figure 3.2 uG24081616uP Instruction Pipeline staging.....	18
Figure 3.3 Bus Interface Unit (BIU) IO diagram.....	20
Figure 3.4 Arithmetic and Logical Unit (ALU) IO diagram.....	22

List of Tables

Table 1.1 Abbreviations & Acronyms	8
Table 1.2 Implementation Nomenclature.....	9
Table 2.1 uG26081616uP Clocks-Resets	11
Table 3.1 12-bit Argument Instruction Format for Rd/Xd instructions	16
Table 3.2 12-bit Argument Instruction Format for Rs/Xs instructions.....	16
Table 3.3 10-bit Argument Instruction Format for immediate instructions	16
Table 3.4 10-bit Argument Instruction Format for Xs instructions	16
Table 3.5 0 Argument Instruction Format	16
Table 3.6 2-fetch Instruction Format	17
Table 3.7 2-operands Instruction Format	17
Table 3.8 Data Transfer Instruction Format.....	17
Table 3.9 Arithmetic Instruction Format	17
Table 3.10 Logical Instruction Format	17
Table 3.11 Pipeline States-Actions	19

Revision History

Rev. No.	Date	Authors	Description
0.1			Initial draft of the Engineering Specification

CONFIDENTIAL

1 Introduction

uG24081616uP is an 8-bit, in-order, single thread, single issue, real-time integer core based on load-store architecture.

1.1 Features

- 16-bit Instruction length for wide operation support
- 4-stage pipelined core for improved timing and performance
 - Instruction Fetch (IF), Instruction Decode (ID), EXecute (EX) & Write Back (WB)
- 16 8-bit General Purpose Registers (GPR)
- 4 16-bit Special Function Registers (SFR); partly controlled through instructions
 - Program Counter (PC)
 - Return Address (RA)
 - Program Status Word (PSW)
 - Stack Pointer (SP)
- Implements AMBAv5 Advanced High-performance Bus (AHB) for memory accesses
- Implements 16-bit address bus i.e. 64KB Total Addressable Memory
- Configurable Tightly Coupled Memory (TCM)
 - 0/1/2/4/8/16KB
 - Memory-mapped, single-cycle
- Reset-strapped PC & SP values
- Implements separate (but synchronous) Core and Bus (AHB-HCLK) clocks-resets
- Configurable Stack sizes of 1/2/4/8KB
- Configurable Debug support through JTAG with 2/4 configurable breakpoints
- Supports single thread, single issue only
- Configurable implementation of Multiply & Divide instructions
 - Configurable support for 1/2/4/8-stage pipelined Multiply
 - Fixed 8-stage Divide
- Supports conditional & unconditional jumps
 - Conditional & Unconditional PC-relative jumps upto +/- 1KB
 - Unconditional Indirect jumps upto +/- 256B
 - Unconditional Absolute jumps
- Configurable implementation of Pre-Fence and Post-Fence instructions
- Supports 2 16-bit Data Pointers and 8 16-bit Instruction Pointers through GPRs
 - Data Pointer selection is embedded in the PSW SFR
 - Instruction Pointer selection is embedded in the instruction
- Implements separate Increment & Decrement instructions with operation size of 1-16
- Implements 2 Interrupts and 3 Exceptions with independent service handler vectors
 - Interrupts: Maskable (MI) & Non-maskable (NMI)
 - Exceptions: Illegal Instruction (IE), Memory Access (ME) & Stack Overflow/Underflow (SE)
- Configurable implementation of performance counters
 - 32-bit Cycle Counter that indicates the life of the core post reset deassertion
 - 32-bit Instruction Counter that indicates the number of instructions retired
- Implements Wake For Interrupt (WFI) instruction for low-power operation of the core

1.2 Scope & Purpose

The scope of this document is limited to the specification of the microprocessor IOs, Configurations, Functionality, Implementation, Programming & Integration. This document doesn't deal with the fundamental concepts of computer organization or core design.

1.3 Abbreviations & Acronyms

Table 1.1 Abbreviations & Acronyms

Abbreviation/ Acronym	Expansion
AHB	Advanced High-performance Bus from the Arm AMBA standard
AMBA	Advanced Microprocessor Bus Architecture
ALU	Arithmetic and Logical Unit
CMU	Clock-Reset Management Unit
CPU	Central Processing Unit
CSR	Control and Status Register(s)
DPTR	Data Pointer
DMA	Direct Memory Access
DMAC	DMA Controller
DTCM	Data TCM. A memory where the data is typically stored.
FIFO	First In First Out
FSM	Finite State Machine
GPIO	General Purpose Input Output
GPR	General Purpose Register
IC	Integrated Circuit
IP	Intellectual Property
IPTR	Instruction Pointer
ITCM	Instruction TCM. A memory where the firmware is typically stored.
PC	Program Counter
PMU	Power Management Unit
PSW	Program Status Word
RA	Return Address
RAM	Random Access Memory. A volatile memory that can be randomly accessed to read or store data.
ROM	Read Only Memory. A memory that can only be read and not written.
RTC	Real Time Clock
SFR	Special Function Register
SoC	System on Chip
SP	Stack Pointer
TCM	Tightly Coupled Memory
WDT	Watch Dog Timer

1.4 References

Below are the references for the development of this Specification document.

1.5 Implementation Nomenclature

Table 1.2 Implementation Nomenclature

Nomenclature	Interpretation
clk	Clock signal
rst	Reset signal
_n/n	At the end of a signal indicates Active LOW Polarity
adr/addr	Address line(s)
dat/data	Data line(s)
cntl/ctrl	Control signal/logic
en	Enable signal
sel	Select signal
intr/irq	Interrupt request/signal/logic
req	Request signal/logic
clr	Clear signal/logic
err	Error signal/logic
fsm	FSM logic
cst	Current State of an FSM
nst	Next State of an FSM
ack	Acknowledge signal
nack/nak	No/Negative Acknowledge
sync	Synchronized signal or Synchronization logic
async	Asynchronous signal
tout	Timeout signal/logic
cmp	Compare result/status
done/cmpl	Event/Task/Process/Operation/Activity is Done/Completed
arb	Arbitration Logic (contextually Grant signal)

2 Top Level Design

2.1 Overview

uG24081616uP microprocessor resides in an SoC and functions as programmed in the firmware. This firmware may reside in the ITCM or an on-chip ROM/RAM or some external memory. While the processor's behaviour (and the associated actions) per instruction is determined, the function of the processor as a system is purely governed by the firmware provided to it. The below diagram illustrates the overview of an SoC with the uG24081616uP microprocessor configured for the full set of features it supports.

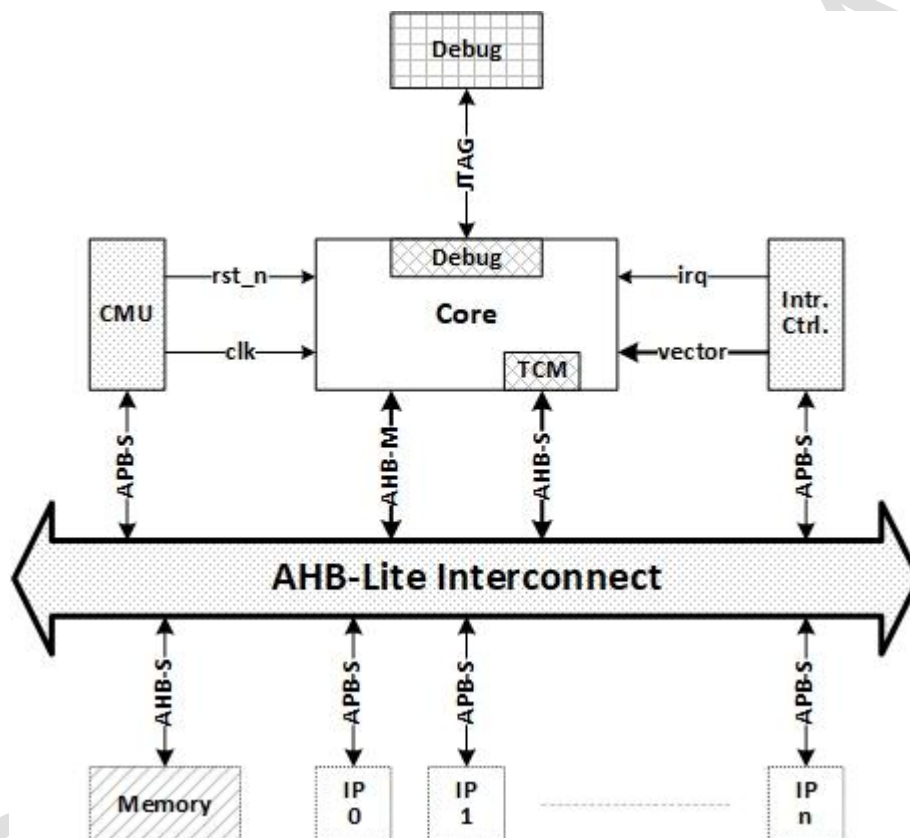


Figure 2.1 uG24081616uP System Overview

Let us understand the various components of an SoC involving the uG24081616uP microprocessor.

The chequered portions of the core are its configurable blocks and exist only if configured for. The AHB-Slave interface of the core is non-functional if the core is not configured to implement any of the TCMs. The JTAG interface of the core is non-functional if the core is not configured to support JTAG Debug. The configuration of these logic partitions is SYSTEM IMPLEMENTATION DEFINED.

The blocks shaded with dots; namely, CMU, Interrupt Controller, AHB Interconnect and a memory (that can be TCM or an on-chip/off-chip ROM/RAM) are the basic building blocks of an SoC and are needed for the core to function. System integrators can do away with them after a thorough analysis but must implement the required supporting logic in their place. The Memory block shaded with slanting lines is optional and may not exist if the system can work with the TCMs or an external memory.

The blocks with dashed line borders are other functional partitions of the system that work along with the core but needn't necessarily affect the functioning of the core directly. Such functional partitions are SYSTEM IMPLEMENTATION DEFINED. RTC, WDT, DMAC, GPIO, System Control Block etc. are some examples (not limited to) of these blocks.

CMU provides the processor with the core & AHB clocks-resets. Once the processor is out of reset and the core clock starts toggling, the processor fetches the first instruction from the memory location pointed by the i_reset_pc input and the instruction fetches thereafter are controlled by the firmware.

The processor services hardware interrupts when any of the interrupt inputs (i.e. i_hw_irq or i_hw_nmi) are asserted HIGH. uG24081616uP only has precise exceptions and force the processor to trap in the case of an exception. The processor services the interrupt from the vector provided for the respective interrupt type. Similarly, processor services encountered exceptions from the respective vectors provided for the respective exception. Redirections may require storing some or all the architectural state into the stack. Ensuring sufficient stack for an application is the system integrator's responsibility.

2.2 Top Level Configurability

Please refer uG24081616uP_Config sheet of the uG24081616uP_TopLevel.xlsx file.

2.3 Top Level IO Description

Please refer uG24081616uP_IO sheet of the uG24081616uP_TopLevel.xlsx file.

2.4 Clock-Reset Specification

This section of the document describes the specification of the Clocks & Resets of the uG24081616uP microprocessor.

uG24081616uP microprocessor has 3 clock-reset inputs.

Table 2.1 uG24081616uP Clocks-Resets

Clock	Reset	Comments
core_clk	core_rst_n	Core Clock-Reset All the core except the BIU and the JTAG-Debug logic operate on this clock-reset.
HCLK	HRESETn	AHB Clock-Reset BIU, TCMs & CSRs operate on this clock-reset. In simple, all the bus related operations take place with this clock-reset.
JTAG_TCK	JTAG_TRSTn	JTAG Clock-Reset All the JTAG-TAP and Debug CSRs operate on this clock-reset. This clock-reset exist only if JTAG is configured to Enabled.

Though uG24081616uP microprocessor has 2 distinct AHB interfaces (1 master for the core and 1 slave for external TCM accesses), implementing independent HCLK-HRESETn inputs per interface increases the area-power consumption (even with an N:1 synchronous relationship) due to multiple domain crossings. So, uG24081616uP operates both the interfaces on the same HCLK-HRESETn.

2.4.1 Clocks

All the sequential logic of uG24081616uP microprocessor (except the JTAG TDO flop) operate on the rising edge of their respective clock.

2.4.1.1 Adaptive Synchronous Handshake

Since significant communication happens between the core_clk and the HCLK domains, having an asynchronous relationship between them leads to more logic for synchronization, thereby increasing the area-power consumption and reducing the performance of the core due to synchronization delays. So, uG24081616uP microprocessor mandates an **N:1** synchronous relationship between the core_clk and the HCLK domains. An N:1 synchronous relationship is permitted because the core may operate at a higher frequency than the bus (i.e. AHB), for various reasons. Here, **N is an integer ≥ 1** . The logic scheme implemented to achieve such N:1 synchronous relationship, where N is not a fixed number, is called adaptive handshaking.

N is a known quantity in a fixed relationship and the logic implementation can be done accordingly. But, such fixed relationships limit the system integrator's ability for low power implementation and so, adaptive handshaking is implemented to save area as well improve low power efficiency of the system.

Since N:1 guaranteed the core clock frequency to be integer multiples of the bus clock, nothing different is done for signals launching in the bus clock and capturing in the core clock because the bus clock is slower (includes equals to) than the core clock. Signals launched in the bus clock are sampled in the core clock on their respective changing edge and since such signals launch as a response to some action initiated from the core clock, the handshake for the signals crossing from bus clock to core clock is straightforward. It is not the case for the signals launching in the core clock and crossing into the bus clock. Here, the assertion of signals in the core clock is dependent on the capturing of its previous value in the bus clock because the core clock is faster (includes equals to) than the bus clock. The following diagram illustrates a typical adaptive handshaking between the core and bus clocks.

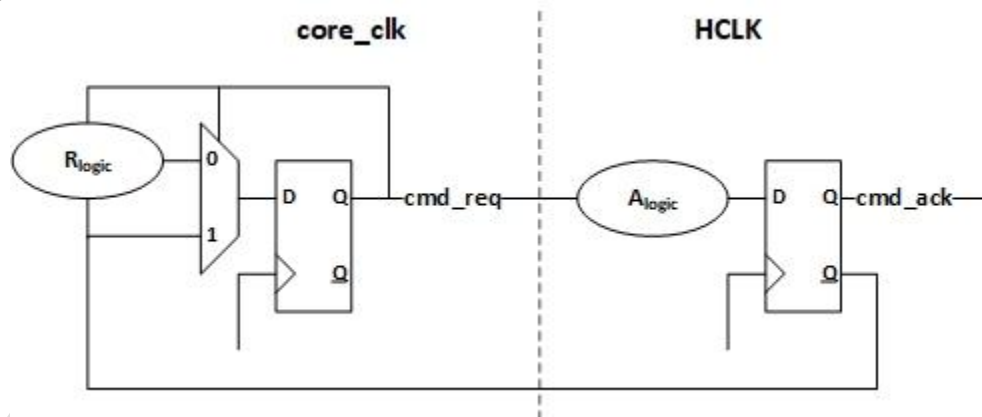


Figure 2.2 Adaptive Synchronous Handshake

In the above illustration, a common request signal (cmd_req) launches in the core clock domain and reaches the BIU, to be captured in the bus clock domain. In an adaptive handshake mechanism, the assertion/de-assertion of the command request is dependent on the capturing of its state in the bus clock domain and its state is unchanged till the capture is done. The command acknowledge signal (cmd_ack) launches in the bus clock domain and reaches the core clock domain to acknowledge the receipt of the assertion/de-assertion of the request.

R_{logic} is a combinational Request logic in the core clock domain responsible for the assertion of the command request while A_{logic} is a combinational Accept logic in the bus clock domain responsible for accepting the command request when the conditions permit it. This way, R_{logic} waits for a LOW state on the command request before asserting the next request (if any) and the A_{logic} can delay the acceptance of a command request in situations like the bus being busy while servicing another (or previous) request.

Refer Implementation Guidelines for implementation specific details.

2.4.1.2 JTAG Adaptive Clocking (if configured)

Since JTAG_TCK is a clock from an external source, it's relationship with the other clocks of the core cannot be determined and is always assumed to be asynchronous. If JTAG TCK is configured to Adaptive Clocking scheme, the synchronized version of the JTAG_TCK is synchronous to the core_clock but the JTAG_TCK is not directly synchronous to the other clocks of the microprocessor.

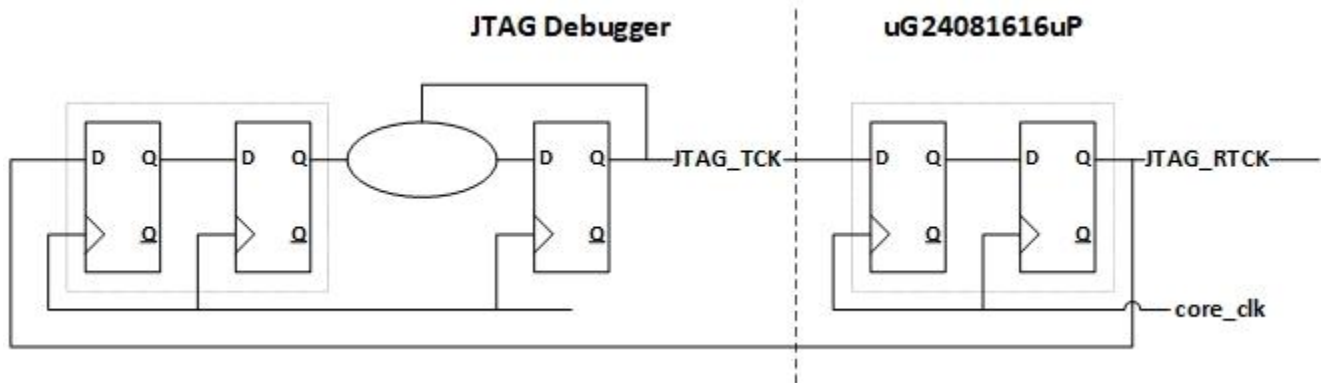


Figure 2.3 Adaptive Clocking scheme

In adaptive clocking, the initiator (JTAG debugger in this case) toggles the JTAG_TCK when there is a transaction to perform with the core and the state of the JTAG_RTCK matches the current state of the JTAG_TCK. JTAG_RTCK is an output of the core and is a synchronized version of the JTAG_TCK to the core clock. This way, both the devices are aligned to operate at the lowest frequency possible due to the synchronization to their respective domain clocks. Synchronization is represented in the dotted line borders in the diagram. While this mechanism is slower, it offers the below advantages over the non-adaptive clocking scheme.

1. Devices are not required to meet timing at a high frequency to make themselves compatible with high performance partners.
2. Since RTCK is used as the clock, additional synchronization schemes aren't needed between the core/bus clock domains and the JTAG Debug domain, thereby reducing area.
3. While performance seems reduced at the JTAG interface, avoiding synchronization delays inside the logic as part of completing a transaction improves the overall performance of the debugging process.

When JTAG TCK is configured for Adaptive Clocking scheme, JTAG Debug logic operates on the JTAG_RTCK, else directly on the JTAG_TCK clock input.

2.4.2 Resets

Of the 3 reset inputs, JTAG_TRSTn launches from an external source whose specification is governed by the IEEE1149.1 standard and cannot be bound by the core specification. uG24081616uP microprocessor is designed such that the JTAG_TRSTn reset operation is maintained independent of the core operations and only limited to the JTAG Debug logic.

Since the core and bus clock domains need to operate concurrently, the behaviour of the Core Reset (i.e. core_rst_n) and Bus Reset (i.e. HRESETn) should adhere to the following specification.

1. Core Reset should de-assert at least 2 core clocks after the de-assertion of the Bus Reset. Since the core clock can be way faster than the bus clock, early wake-up of the core reset can cause the core to place a transfer request while the bus clock logic is not out-of-reset and may lose the request. Adhering to this specification ensures a proper receipt of the request. Though adaptive handshaking avoids loss of such requests, it is recommended to adhere to this specification.

2. While the core reset can alone (since bus is a shared resource) be asserted, it's de-assertion should always adhere to the above specification.
3. In situations where the bus reset is alone asserted, the system integrator should ensure no transfers requested by the core logic are pending on the bus. In situations of pending requests/transfers on the bus and the system is stuck, the system integrator cannot reset the bus alone and the core should also be reset for a proper recovery of the complete system.
4. If the system state is such that the bus reset can alone be asserted, it is recommended (not mandatory) to gate the core clock. System integrators should exercise discretion and caution while choosing an approach in the interest of the system requirements.

Since the CMU is implemented outside of the uG24081616uP microprocessor, it is the system integrator's responsibility to ensure proper functional timing of the clocks and resets.

3 Design Specification

uG24081616uP microprocessor is one of the key functional partitions of an SoC that functions as programmed in it's firmware. It is initialized by 2 resets and operates on 2 synchronous clocks.

Once the core is out of reset and the clock starts to toggle, it fetches instructions from the address strapped to the i_reset_pc input of the core and the subsequent fetches are controlled by the firmware. The fetched instructions are decoded and executed as per the ISA.

- The processor can be redirected to service interrupts. uG24081616uP only has precise exceptions and force the processor to trap in the case of an exception. Redirections may require storing some or all the architectural state into the stack. Ensuring sufficient stack for an application is the system integrator's responsibility.
- uG24081616uP supports configurable pipelining and if configured to support, implements 4 pipeline stages; namely, IF (Instruction Fetch), ID (Instruction Decode), EX (Execute) & WB (Write Back). Further section(s) in this chapter of the document elaborates more on the pipelining.
- uG24081616uP supports configurable debug and if configured to support, implements JTAG Debug which enables the programmer to halt the core at the debugger programmed breakpoint and view the processor architectural state.
- uG24081616uP supports configurable divide instruction with an 8-cycle execution. uG24081616uP supports configurable pipeline stages for multiply instruction (if configured to support the multiply instruction). Both these instructions are out-of-pipe.
- uG24081616uP supports configurable fence instructions which stall the processor pipelines accordingly and are used to create memory access ordering and boundaries.
- uG24081616uP implements a single AHB-Master interface (i.e. AHB-M) for both the instruction and data transfers while transfers to the TCMs (if configured to exist) are resolved inside the core and performed directly. This is done to improve system performance. Transfers from external agents to the TCMs are performed through a dedicated AHB-Slave interface (i.e. AHB-S) and cannot access any other region of the core other than TCMs.

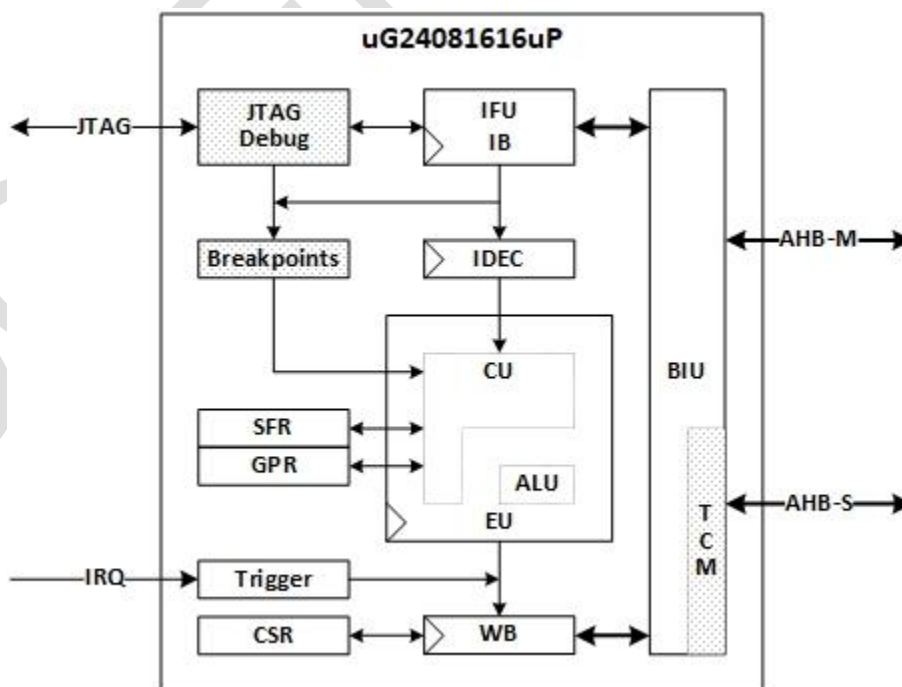


Figure 3.1 uG24081616uP Microarchitecture diagram

The above diagram represents the microarchitecture of the uG24081616uP microprocessor. Blocks with an angled brace “>” indicate a pipeline stage in that block. Blocks shaded with dots exist only if configured to. The direction of the arrows in the diagram indicates the flow of significant information and does not limit to only that direction of information flow. Further sections in this chapter of the document elaborate more on the functional partitions of the core. For a better understanding of the various functional blocks, it is important to understand few fundamentals of the core. The following sections preceding the functional partitions discuss those fundamental aspects of the core.

3.1 Instruction Set Architecture

This section of the document describes the instruction set and its decode related aspects of the uG24081616uP microprocessor.

3.1.1 Instruction Set

Please refer uG24081616uP_ISA sheet of the uG24xx1616uP_ISA.xlsx document for the detailed specification of each instruction supported by the uG24081616uP microprocessor.

3.1.2 Instruction Formats

This section of the document describes the instruction formats that enables the understanding of the IFU & cross-compiler developers.

The instruction length of the uG24081616uP microprocessor being 16-bit, the instruction formats are compelled to be designed with argument size in view for the efficient management of the instruction bits and the execution of the instructions in a single fetch.

3.1.2.1 12-bit Argument Instruction Format

All immediate and Load-Store Data Transfer instructions fall into this category.

Table 3.1 12-bit Argument Instruction Format for Rd/Xd instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i8								Rd/Xd				func			1

Table 3.2 12-bit Argument Instruction Format for Rs/Xs instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Rs/Xs				i8								func			1

3.1.2.2 10-bit Argument Instruction Format

All Control Transfer instructions except Absolute Jump instructions fall into this category.

Table 3.3 10-bit Argument Instruction Format for immediate instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i10										func				1	0

Table 3.4 10-bit Argument Instruction Format for Xs instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i7							Xs				func			1	0

3.1.2.3 0 Argument Instruction Format

Instructions with no arguments fall into this category.

Table 3.5 0 Argument Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
func								0000				00		0	0

NOP, WFI, RET and Fence instructions fall into this category.

3.1.2.4 2-fetch Instruction Format

Only Absolute Jump Instructions fall into this category. Since uG24081616uP has 16-bit addressing, absolute instructions cannot be accommodated in a single instruction fetch and so, these instructions require to fetch the next instruction in the memory space for the 16-bit address.

Table 3.6 2-fetch Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	func				00		0	0
a16															

Here, func[3:1] is 0b100.

3.1.2.5 2-operands Instruction Format

This category of instructions has 2 operands and no/fixed destination register. The operation of these instructions requires 2 operands and operate directly over them or into fixed destination register(s). MUL & DIV instructions fall into this category and operate over fixed destination registers.

Table 3.7 2-operands Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Rs1/Xs1				Rs2/Xs2				func				00		0	0

NOTE: Rs1 & Rs2 can also mean valid SFRs in the case of Data Transfer instructions.

Here, func[3] is 0b0 and func[2:0] is a non-zero value.

3.1.2.6 Data Transfer & Stack Instruction Format

All data transfer instructions that do not belong to the above instruction formats and all stack instructions fall into this category.

Table 3.8 Data Transfer Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Rs/Xs				func				Rd/Xd				01		0	0

NOTE: Since Stack operations either write or read architectural state but not both in the same instruction, either Rs/Xs or Rd/Xd exists for a specific instruction. The non-existent field of that specific instruction will be 0s in that case.

3.1.2.7 Arithmetic & Logical Instruction Format

All Arithmetic & Logical instructions that do not belong to the above instruction formats fall into this category. MUL & DIV don't fall in this category but into the 2-operands instruction format.

Table 3.9 Arithmetic Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Rs				func				Rd				10		0	0

Table 3.10 Logical Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Rs				func				Rd				11		0	0

NOTE: Rs bit positions would be used as Auxiliary immediate for some instructions.

3.2 Instruction Pipeline Staging

uG24081616uP microprocessor implements a 4-stage pipeline, when HAS_PIPELINES is configured to implement pipelines. The 4 stages of pipelines are Instruction Fetch (IF), Instruction Decode (ID), Execute (EX) and Write Back (WB). The processor implements only the IF & WB stages when not configured to implement pipelines. The reason for implementing the IF & WB stages for a non-pipelined configuration is explained in the later sections of this section of the document. When HAS_PIPELINES

is configured False, the outputs of the ID & EX stages are not registered but the outputs exist and are passed on to the subsequent blocks till retirement. The following diagram illustrates the instruction pipeline staging when HAS_PIPELINES is configured True.

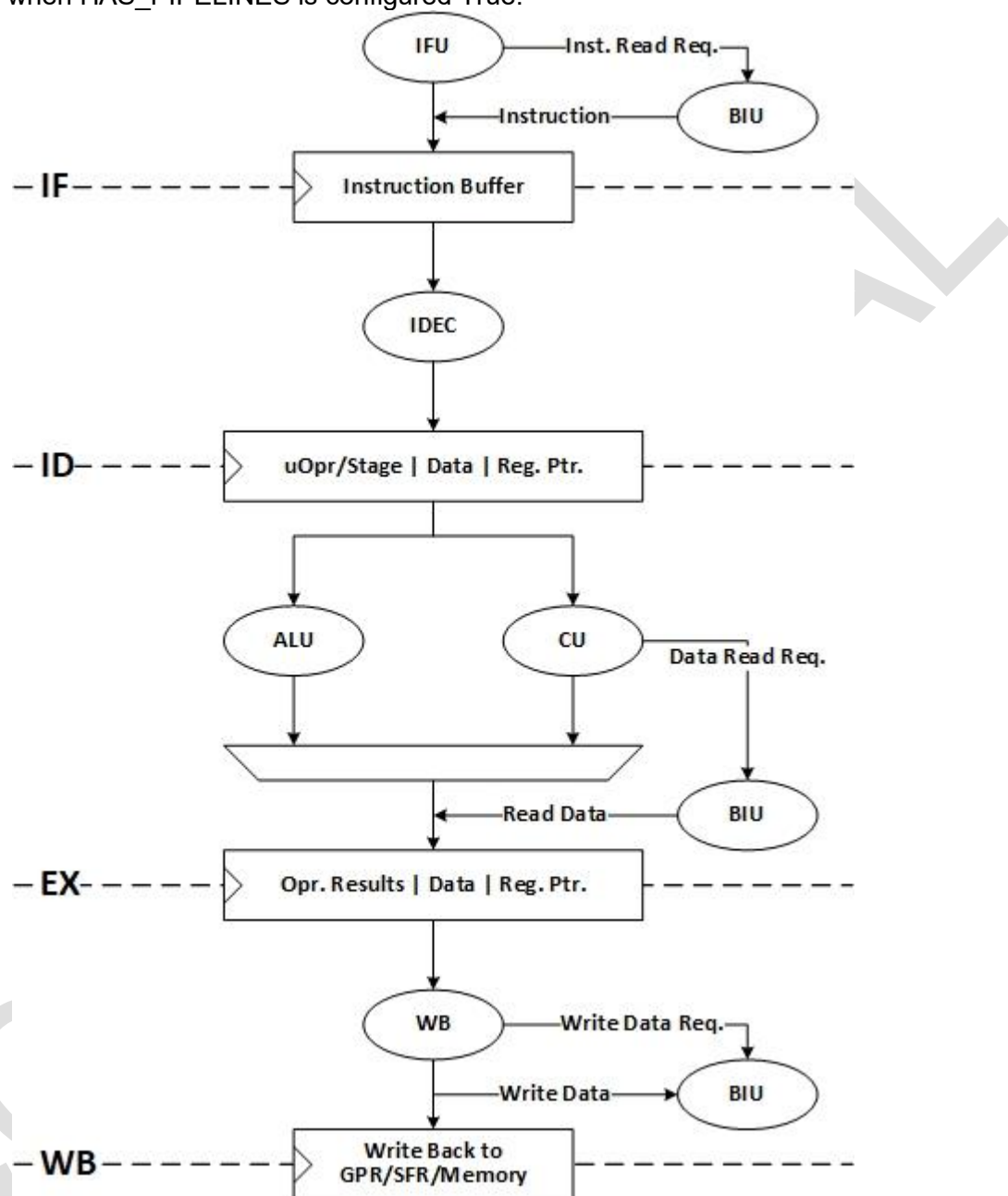


Figure 3.2 uG24081616uP Instruction Pipeline staging

Dashed lines indicate the name of the stage and box indicates the information registered in that pipeline stage. The angled brace ">" indicates the registering of information.

3.2.1 Instruction Fetch (IF)

This stage is created to buffer the instructions received on the bus. In simple, the output of this stage is the instruction to be decoded. The registers for this stage physically exist in the IFU block. This buffering is inevitable for the below 2 reasons.

1. Buffering the instructions from the bus will improve timing on the bus side as well the decode.

2. Since uG24081616uP implements 2-fetch instructions, buffering enables for better routing or bypassing the instruction decode.

3.2.2 Instruction Decode (ID)

This stage registers the decoded instruction's control information to the subsequent stages. The output of this stage provides the micro-operational details of the instruction to the subsequent stages and all the blocks involved till the retirement of an instruction. The registers for this stage physically exist in the IDEC (i.e. Instruction Decoder) block. This stage exists only when configured to implement pipelining.

3.2.3 Execute (EX)

This stage registers the executed results of an instruction as directed by the decoder. The outputs of this stage are either the output of an ALU operation or the data read from the memory/GPR/SFR or control information passed down to the write back. In simple, the outputs of this stage are the executed results of the instruction. The registers for this stage physically exist in the Execute block. There are 2 sources to this staging; namely, Control Unit and ALU. ALU forms the source for all the Arithmetic and Logical instructions. Control unit forms the source for all the other instructions. Control Unit forms the source for instructions requiring the usage of the ALU (ex. load-store instructions with address offset) but not falling into the Arithmetic or Logical instruction groups. This stage exists only when configured to implement pipelining. Any instruction crossing this stage is said to be COMMITTED.

3.2.4 Write Back (WB)

This stage ensures writing back of the values into the GPRs/SFRs/CSRs/Memory. This being an instruction timing stage, the output of this stage is the respective destination and not physical registers in the Write Back block. Execute is a huge process and writing directly into the GPRs/SFRs/CSRs/memory can adversely affect timing and so this stage is inevitable. Instructions crossing this stage are said to be RETIRED.

3.3 Pipeline States-Actions

In the broader process of fetch-decode-execute, the pipeline stages act as a perennial source of instructions in the different phases of a CPU process. When a CPU is staged as pipelines, the behaviour a stage varies based on it's current stage of operation. This section of the document deals with the different states of the pipeline stages and their actions because of those state controls.

Table 3.11 Pipeline States-Actions

State	Description
Valid	Valid indicates the stage that the information available with it is valid and needs to be processed. The opposite of this is invalid and the stage remains idle (i.e. performs no actions) till it is moved to Valid state. A stage cannot place itself in Valid state. An older stage does this. The order is IF->ID->EX->WB.
Halted	Halted indicates the stage to have had Valid information to process but prevented by external control(s). Halting can happen through debug calls and fence instructions. Halting the older stages automatically cause the younger stages to become invalid as the process proceeds forward.
Stalled	Stalled indicates the stage to have had Valid information to be process but prevented by external situation(s). Stalling can happen through memory access waits.
Killed	Killed indicates the stage to have Valid information to process but is invalidated due to a change in the course of execution. Killing can happen due to a redirection in the program like an unconditional branch or a taken branch, context change due to a trap etc.

Killed > Stalled > Halted > Valid is the order of priority of the above states.

Valid state of a stage is dependent on the state of the younger and older stages. The implementation of Valid per stage is handled by 2 conditions per stage, namely; done and next valid. Done indicates the completion of processing an instruction in that stage. Next valid of a stage indicates the validity of the next instruction entering the stage. Next valid and done prevail in the absence of Halted/Stalled/Killed.

The done of a stage is set when the instruction in that stage is processed. It is cleared when the current stage receives a valid instruction from it's older stage or a done from it's younger stage. Set prevails over the clear in situations where both are asserted. Next valid is asserted when an instruction from it's older stage is processed (i.e. both the valid and done are asserted).

Halted and Stalled states are simultaneous to Valid as these conditions are external and will make the state Valid once the conditions are fulfilled. But Killed is a state that dismisses the other states. This leads to a misnomer that Killed is exactly Not Valid. Killed is synonymous to Invalid but not exactly true for the below reasons.

1. Since Valid is a registered state (that too one-hot), it takes a clock cycle to clear by when the respective instruction in the pipelines is processed.
2. Though the pipeline is Killed, the IFU still functions to fetch the instructions from the redirected PC and the first instruction in the IBUF will have to assert a Valid for the IF stage to proceed the redirection.
3. Unconditional branches cause the younger pipeline stages to be Killed but the older stages continue to be Valid and process the instructions till retirement.

3.4 Bus Interface Unit (BIU)

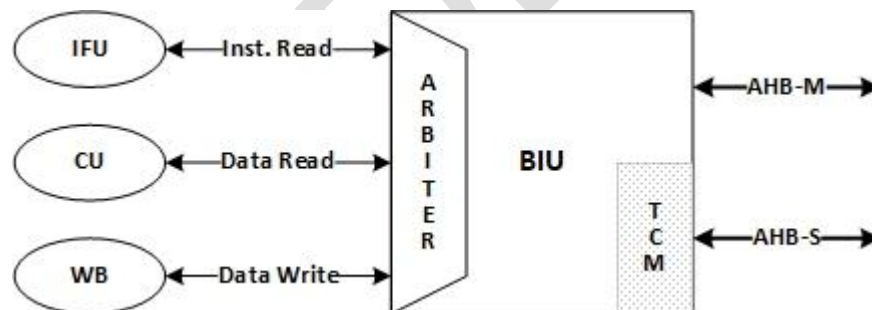


Figure 3.3 Bus Interface Unit (BIU) IO diagram

This functional partition is responsible for accepting transfer requests from the various components of the core and place them on the bus. In simple, this block manages the transfers on the bus. TCMs are shaded to indicate that their existence is based on the configuration of the ITCM_SIZE and DTCM_SIZE parameters. TCMs are single-port memories and this implementation choice is made to reduce the area of the core. The behaviour of the BIU is described below.

- BIU accepts transfers from 3 agents, namely; IFU, CU & WB.
 - IFU issues instruction read requests only.
 - CU issues data read requests only
 - WB issues data write requests only
- BIU implements fixed priority arbitration in the below order of priority
 - Data Read requests
 - Instruction Read requests
 - Data Write requests.

- HPROT[0] is asserted HIGH for Data Read-Write request transfers and LOW for Instruction Read request transfers. HPROT[1] (i.e. Privileged access) can be any fixed value based on the integrator's choice and is agnostic of the request interface.
- BIU resolves if a transfer request is to the bus or to the TCMs (if configured to implement) and accordingly services the transfer request. Transfer requests accessing the TCMs are not placed on the AHB-M interface but rather serviced directly with the TCMs.
- BIU implements fixed priority arbitration in the below order of priority. This is done to avoid stalling the core for prolonged periods and since TCMs are filled by an external agent like a DMA controller, those transfers can wait compared to the internal accesses.
 - Transfer agent (IFU/CU/WB) requests
 - AHB-S interface bus requests
- BIU operates in the HCLK domain and implements the Accept logic mentioned in the Adaptive Synchronous Handshake section of the document on each transfer agent path with the respective Request logic implemented in the respective transfer agent. In the case of Data Write requests, data valid (from WB) and data acknowledge (from BIU) handshake is implemented as data transfers happen after the command is accepted by the BIU. In simple, the whole purpose of adaptive synchronous handshaking is to ensure lossless data transfers in the fastest possible way.
- The transfer request interface from the agents typically looks like below.
 - Command: cmd_vld, cmd_ack, command - start address, burst length (1-16), access (write/read), transfer type (instruction/data)
 - Data Write: data_vld, data_ack, data
 - Data Read & Response: resp_vld, response (Okay/Error), data

3.5 Instruction Fetch Unit (IFU)

This functional partition is responsible for fetching instructions from the Program Counter and stores them in the instruction buffer. uG24081616uP implements speculative instruction fetches for the efficient use of the bus (i.e. AHB). It is also responsible for tracking the PC offset from the currently servicing instruction for relative jumps. It is also responsible for flagging for the memory exception in case of an error response for an instruction fetch.

3.5.1 Instruction Buffer (IBUF)

Instruction buffer is a 4-deep 16-bit wide pre-emptive FIFO with flush support to flush the fetched instruction in case of a redirection (i.e. Killed state). A FIFO is chosen instead of a simple instruction register for the following reasons.

1. uG24081616uP performs speculative instruction fetches, a FIFO (at least 4-deep) would be efficient for the bus accesses than a single register.
2. uG24081616uP ISA implements 2-fetch instructions and a FIFO would be more effective to mark the current instruction out of the buffer as data (i.e. not to be decoded).

3.6 Instruction Decoder (IDEC)

This functional partition is responsible for decoding the instructions into micro-operations. It is also responsible for flagging the invalid instruction exception when an invalid instruction is decoded.

3.7 Execution Unit (EU)

This functional partition is responsible for the execution of the decoded instructions.

3.7.1 Arithmetic and Logical Unit (ALU)

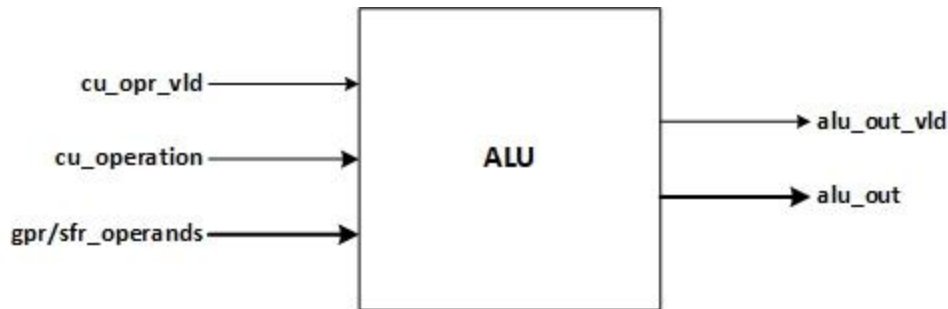


Figure 3.4 Arithmetic and Logical Unit (ALU) IO diagram

This functional partition is responsible for the execution of the arithmetic and logical operations of a decoded instruction.

3.7.2 Control Unit (CU)

This functional partition is the master of the whole uG24081616uP microprocessor and is responsible for controlling the pipeline.

3.8 Write Back

This functional partition is responsible for any writing back operations of a decoded instruction.

3.9 GPRs, SFRs & CSRs

Please refer uG24081616uP_Registers sheet of the uG24xx1616uP_ISA.xlsx document for the detail specification on GPRs and SFRs.

Please refer PSW sheet of the uG24xx1616uP_ISA.xlsx document for the detailed specification of the Program Status Word SFR.

The specification for the CSRs is not yet clear and will update it along with the JTAG Debug.

3.10 JTAG Debug

The specification for the JTAG Debug is not yet clear and will update it later.

3.11 Reset Strapping

uG24081616uP implements reset strapping on the PC & SP SFRs i.e. the default value of these SFRs on reset are determined by the i_reset_pc and i_reset_sp inputs respectively. PC and SP SFRs are loaded with the values from the i_reset_pc & i_reset_sp inputs respectively on the first rising edge of the core clock and once the core is out of reset. The core fetches instructions from second rising edge of the core clock onwards, from the PC value loaded in the first rising edge of the core clock.

3.12 Performance Counters

uG24081616uP implements the below 2 performance counters when HAS_HPC is configured HIGH.

1. **Machine Cycles:** It is a 32-bit counter that counts the number of clocks cycled by the core once it is out of reset. On reset, this counter is initialized to 0.
2. **Instructions Retired:** It is 32-bit counter that indicates the number of instructions retired by the core once it is out of reset. On reset, this counter is initialized to 0.

The values in these performance counters can be read through the respective CSRs allocated for them.

4 Appendix

This section of the document describes the implementation specific requirements & limitations for external integrators & programmers to use the block properly.

4.1 Integration Guidelines

4.2 Programming Guidelines

4.3 Implementation Guidelines

This section of the document describes the implementation guidelines of the uG24081616uP.

1. For the proper distinction of the signals/functions per stage of the core, the signal/function names should have the stage name suffixed. Signals/functions that pertain to the pipeline staging but does not bind to a particular stage should be suffixed with “_xx”. For ex.: valid_id, done_ex, halt_xx etc.
2. Implementers should take care of the N:1 multi-cycle path timing relationship between the core & bus clocks in the functional SDC. This relaxes the timing paths and improves the timing.
3. Implementers should constrain the pipeline retiming for the division path as the design implements a fixed 8-stage pipeline for division. Similarly, pipeline retiming constraints should be implemented in the functional SDC for the multiply path as per the MUL_STAGES configuration.

4.4 Memories

uG24081616uP microprocessor will implement memories if configured to implement an instruction and/or data TCM.

4.5 Macros (Hard IPs)

uG24081616uP microprocessor doesn't require to implement any hard macros (other than for the TCM memory macros) for it's functioning.

Implementing memory macro(s) for the TCM(s) is implementer's choice while uG24081616uP gives provision to replace the generic memory implementation with a macro. Please refer Implementation Guidelines for more details in this regard.

4.6 Performance Analysis

4.7 Estimates

Open Issues

Tabulate any open issues here.

No.	Issue	Comments	Status

CONFIDENTIAL